

server-state

역할: FARM/LAB GPU 서버가 모두 같은 공통 설정을 유지하도록 기준을 정의하고, 운영 서버 점검과 신규 서버 구축에 같은 순서를 사용한다.

설계는 이 모듈이 확인·설정하는 범위, profile 구조, 모듈별 소유권과 현재 구현 수준을 설명하고, **운영**은 점검·구축 흐름에 실제로 사용하는 CLI 명령과 공통 설정 추가 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	확인/설정 범위, profile 구조, 모듈별 소유권, 상태 표시, 디렉터리 구조, 현재 구현 수준
운영	운영 서버 점검·신규 서버 구축 흐름, 공용 서버 목록, 공통 설정 추가 방법, 테스트

처음 보는 사람은 **설계 문서**에서 이 모듈이 무엇을 소유하고 무엇을 다른 모듈에 위임하는지 먼저 확인하고, 실제로 서버를 점검하거나 새 서버를 구축해야 할 때 **운영 문서**를 확인한다.

server-state 설계

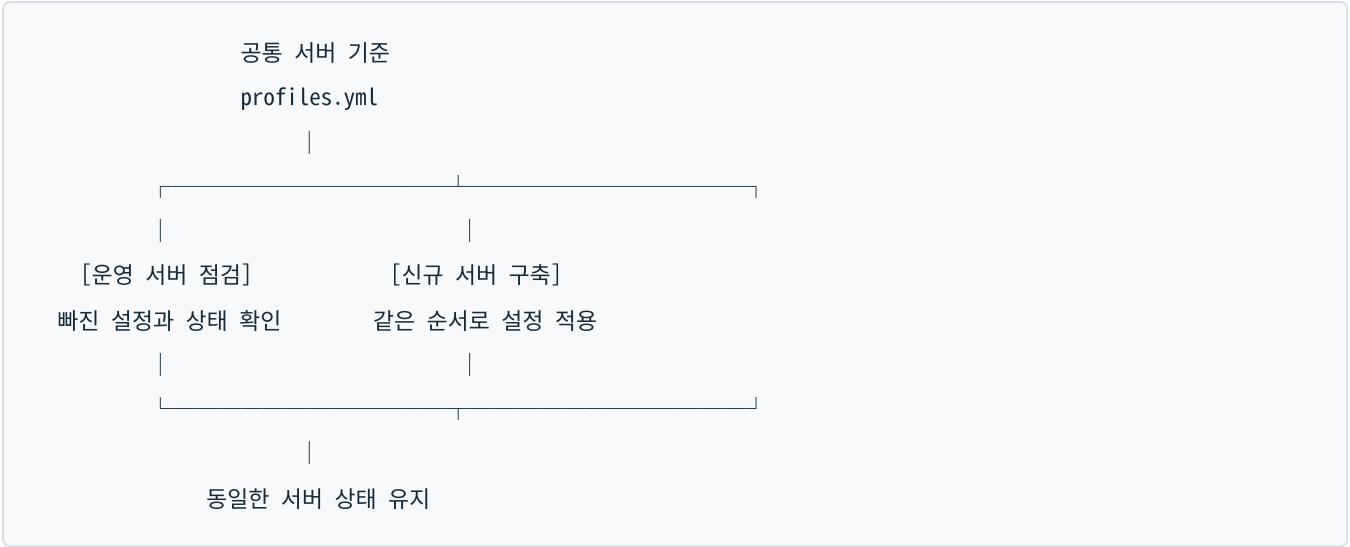
개요 · 운영

1. 이 모듈이 하는 일

server-state는 FARM/LAB 서버가 모두 같은 공통 설정을 유지하도록 기준을 정의하는 모듈이다. 목적은 크게 두 가지다.

- 운영 서버 점검:** 각 서버에 Docker, NVIDIA driver/toolkit, Kubernetes, network tuning, Kerberos/NFS, monitoring 설정이 빠짐없이 적용되어 있는지 확인하고 차이가 있으면 복구할 방법을 제시한다.
- 신규 서버 구축:** 새 서버에 공통 설정을 같은 순서로 적용하여 운영 서버와 동일한 표준 상태로 만든다.

즉, 서버마다 설치 방법을 다시 기억해서 수동으로 설정하는 대신 하나의 공통 기준을 사용하려고 만든 코드다.



`server-state` 가 모든 기능을 직접 구현하는 것은 아니다. 공통 OS, Docker, NVIDIA와 network 설정은 직접 관리하고, Kerberos/NFS와 monitoring처럼 별도 모듈이 소유한 기능은 해당 모듈의 점검·복구 명령을 한 순서로 연결한다.

2. 디렉터리 지도

경로	핵심 기능
<code>README.md</code>	모듈 개요와 책임 범위 요약
<code>bin/server-state</code>	CLI 실행 파일
<code>script/cli.py</code>	<code>list-hosts</code> , <code>check</code> , <code>plan</code> , <code>apply</code> 출력
<code>script/inventory.py</code>	공용 JSONL inventory 로드와 host 선택
<code>script/profiles.py</code>	profile set 확장과 서버별 변수 치환
<code>script/__main__.py</code>	<code>python -m script</code> 로 CLI를 실행할 수 있게 하는 진입점
<code>config/profiles.yml</code>	공통 상태, 점검과 복구 명령의 기준
<code>ansible_playbook/bootstrap_gpu_server.yml</code>	신규/운영 서버의 공통 설정 task
<code>ansible_playbook/kerberos_nfs_client_recovery.yml</code>	Kerberos NFS 상태 점검과 제한된 복구
<code>docs/standard-gpu-server-pipeline.md</code>	신규/운영 서버 표준 단계의 개발 문서
<code>docs/module-boundaries.md</code>	모듈별 소유권 경계 정의 (§ 6 "모듈별 소유권"의 원본 문서)
<code>docs/repo-inventory-cleanup.md</code>	레포 내 일회성/legacy 파일 정리 감사 기록
<code>docs/reusable-operational-files.md</code>	재사용 가능한 운영 파일 목록
<code>requirements.txt</code>	Python 의존성 (<code>PyYAML</code>)
<code>tests/</code>	inventory와 profile 처리 unit test

3. 무엇을 확인하고 설정하는가

`new-host-bootstrap` 과 `existing-host-drift` 는 다음 항목을 같은 순서로 사용한다. 따라서 새로운 공통 설정을 추가할 때 두 흐름에 함께 넣을 수 있다.

순서	영역	운영 서버에서 확인하는 것	신규 서버에 설정하는 것
1	기본 접속 조건	inventory 등록, Ansible 접속, 비대화형 sudo, hostname	자동 설정 전 SSH, sudo, hostname, IP와 inventory를 확인
2	공통 OS	Ubuntu 계열 여부, 공통 package 설치 여부	apt repository, NFS, Kerberos와 network 도구 설치
3	Docker Engine	service 활성 상태, daemon 응답, systemd cgroup driver	Docker repository/package와 <code>daemon.json</code> 설정
4	NVIDIA driver	<code>nvidia-smi</code> 가 GPU와 driver version을 정상 출력하는지, package hold 여부	설정된 driver package 설치와 apt hold
5	NVIDIA Container Toolkit	<code>nvidia-ctk</code> , Docker NVIDIA runtime, containerd NVIDIA runtime	toolkit 설치 후 Docker/containerd runtime 설정
6	Kubernetes node	kubeadm/kubelet/kubectrl, kubelet, cluster join 파일과 node label	Kubernetes package 설치와 kubelet 활성화
7	network tuning	storage NIC 정보, RX queue 4096 이상, 영속화 service	storage NIC RX queue를 4096으로 유지하는 systemd service
8	Kerberos/NFS	realm 설정, machine keytab, service ticket, <code>rpc-gssd</code> , fstab과 mount 상태	client package/config와 GSS 준비 상태 구성
9	monitoring	두 exporter service와 metrics/health endpoint	monitoring 모듈의 exporter 배포 playbook 사용
10	사용자 container 전제조건	사용자 DB 환경, server inventory, Docker 접근	사용자 생성·삭제는 <code>user-lifecycle</code> 을 통해 처리

3.1 NVIDIA version 확인 범위

현재 점검 명령은 `nvidia-smi` 를 실행하여 GPU 이름과 실제 driver version이 출력되는지 확인한다. 따라서 driver가 GPU를 인식하지 못하거나 명령 자체가 실패하는 상태는 찾을 수 있다.

신규 설치 playbook의 기본 package는 현재 `nvidia-driver-580` 이며 설치 후 의도하지 않은 major version 변경을 막기 위해 apt hold한다. 다만 운영 서버의 driver version을 `580` 과 자동 비교하여 불일치 판정을 내리는 규칙은 아직 없다. 현재는 출력된 version을 관리자가 확인해야 한다.

3.2 network 설정 범위

현재 `network-tuning` 이 관리하는 대상은 스토리지 통신에 사용하는 NIC의 RX queue 크기다. inventory에서 서버별 storage interface를 읽고, RX queue가 4096 이상인지와 `decs-rx-queue.service` 가 활성화되어 있는지 확인한다.

IP 주소, gateway, DNS와 netplan 전체를 자동으로 설정하는 기능은 아직 없다. 신규 서버의 IP, hostname, SSH와 sudo는 bootstrap을 시작하기 전에 준비해야 하는 조건이다. 향후 모든 서버의 netplan까지 통일하려면 별도의 profile과 검증 규칙을 추가해야 한다.

4. 현재 구현 수준

이 모듈은 최종적으로 공통 기준과 실제 서버 상태를 자동 비교하고 필요한 복구까지 안전하게 실행하는 것을 목표로 한다. 현재 구현 수준은 다음과 같다.

기능	현재 상태
전체 서버가 따라야 할 공통 profile 정의	구현됨
신규/운영 서버에 같은 profile 순서 사용	구현됨
서버별 점검 명령 생성	구현됨
신규 서버 설정용 Ansible task	구현됨
check가 원격 서버를 순회하고 결과 판정	아직 구현되지 않음
apply --execute로 복구 자동 실행	아직 구현되지 않음. 명시적으로 거부됨
주기적인 전체 서버 drift 검사와 알림	아직 구현되지 않음
IP/DNS/netplan 전체 표준화	아직 구현되지 않음

따라서 “모든 서버가 같은 설정인지 확인하고 새 서버를 똑같이 설정한다”는 이해는 맞다. 다만 현재는 **기준, 점검 명령, 복구 playbook을 한곳에 정리한 단계**이고, 한 명령으로 전체 서버를 자동 검사·복구하는 controller까지 완성된 상태는 아니다.

5. profile 구조

config/profiles.yml에는 작은 단위의 profile과 이를 순서대로 묶은 profile set이 있다.

profile set	용도
new-host-bootstrap	신규 SSH-ready 서버의 표준 구축 순서
existing-host-drift	운영 서버의 공통 설정 점검·복구 순서
managed-host	운영 관리 서버용 기본 별칭
monitoring-host	monitoring 항목만 확인할 때 사용

새 공통 설정을 모든 서버에 적용하려면 작은 profile로 추가한 뒤 new-host-bootstrap과 existing-host-drift 양쪽에 넣는다. 이렇게 해야 새 서버 설치에는 들어갔지만 운영 서버 점검에서는 빠지거나, 그 반대가 되는 문제를 줄일 수 있다.

각 profile은 다음 정보를 가진다.

- 이 설정을 실제로 소유하는 모듈
- 부작용 없이 상태를 읽는 check
- 차이가 있을 때 사용할 remediation 명령
- 자동 실행할 수 없는 작업의 runbook과 safety level

6. 모듈별 소유권

영역	실제 소유자	server-state 의 역할
공통 OS, Docker, NVIDIA, Kubernetes package, network tuning	server-state	공통 기준과 bootstrap task 관리
NAS, AD, Kerberos와 NFS 정책	kerberos-nfs	점검 순서와 승인된 runbook 연결
exporter와 metrics endpoint	monitoring	배포·점검 playbook 연결
사용자와 container 생성·삭제	user-lifecycle	공용 inventory 사용과 전제조건 확인
서버 전원과 boot sequence	remote-operations	이 모듈에서 다루지 않음
사용자 container image	container-images	이 모듈에서 image 내부를 변경하지 않음

소유권을 나눈 이유는 `server-state` 에 모든 운영 코드를 복사하지 않고, 실제 담당 모듈의 rollback과 안전 절차를 그대로 사용하기 위해서다.

7. 상태 표시

상태	의미
OK	inventory나 로컬 파일처럼 즉시 확인 가능한 조건이 충족됨
MISSING	필요한 로컬 파일이나 값이 없음
DRY-RUN	실행할 원격 점검 또는 복구 명령만 표시함
MANUAL	도메인 가입, join, mount처럼 담당자 확인이 필요한 작업
UNKNOWN	아직 지원하지 않는 check 또는 remediation 형식

server-state 운영

개요 · 설계

1. 운영 서버 점검 흐름

1.1 existing-host-drift 란?

`existing-host-drift` 는 `config/profiles.yml` 에 정의된 **점검 항목 묶음의 이름**이다. 그 안에는 접속 확인(baseline-host)부터 `os-common`, `docker-engine`, `nvidia-driver`, `nvidia-container-runtime`, `kubernetes-node`, `network-tuning`, `kerberos-nfs-client`, `monitoring-exporters`, `user-container-host`까지 10개 영역이 순서대로 들어 있다. "운영 중인 FARM/LAB 서버가 이 10개 영역의 공통 기준을 다 지키고 있는지 확인하는 점검표"라고 보면 된다.

실제로 실행하는 스크립트는 `./server-state/bin/server-state` 이고, 내부 동작은 `script/cli.py` 가 맡는다. `cli.py` 가 `config/profiles.yml` 을 읽어서 `existing-host-drift` 에 묶인 10개 영역의 점검·복구 항목을 순서대로 꺼내 쓰는 구조다.

1.2 "명령어를 만들어서 보여준다"는 게 무슨 뜻인가

`profiles.yml` 에는 완성된 명령이 아니라, `{host}` 나 `{repo}` 같은 자리표시자가 들어간 **명령 틀**만 적혀 있다. 예를 들어 `os-common` 영역의 점검 항목 하나는 실제로 다음과 같이 정의되어 있다.

```
# config/profiles.yml (os-common 영역의 점검 항목 중 하나)
command: "ANSIBLE_CONFIG={repo}/monitoring/ansible_playbook/ansible.cfg \
  ansible {host} -b -m shell -a '. /etc/os-release && test \"$ID\" = ubuntu ...'"
```

`check` 명령을 실행하면 `script/cli.py` 가 이 틀의 `{host}` 를 실제 서버 이름(`farm8`)으로, `{repo}` 를 실제 저장소 경로로 바꿔 채운 뒤, 그 결과 문자열을 화면에 출력만 한다. 예시:

```
$ ./server-state/bin/server-state check --hosts farm8 --profile os-common
farm8 [os-common] ubuntu-release: DRY-RUN
  detail: ANSIBLE_CONFIG=/home/jy/server_manage/monitoring/ansible_playbook/ansible.cfg ansible farm8 -b -m
  shell -a '. /etc/os-release && test "$ID" = ubuntu && printf "%s\n" "$VERSION_ID"
```

`detail:` 뒤에 나온 문자열이 "만들어서 보여준 명령"이다. `server-state` 는 자리표시자를 채운 문자열을 Python에서 만들어 출력하기만 할 뿐, 이 명령을 실제로 실행(예: `subprocess` 호출)하지 않는다. 서버 상태를 실제로 확인하려면 관리자가 이 문자열을 복사해서 직접 실행해야 한다.

1.3 전체 점검 흐름

아래 5단계 중 실제로 서버 상태가 바뀌는 건 5번뿐이고, 그마저도 관리자가 직접 실행해야만 일어난다.

1. **점검 대상 서버 선택** — 공용 inventory(3절 "공용 inventory와 서버 목록" 참고)에서 `--hosts` 조건(`all`, `farm8` 등)에 맞는 서버를 로컬에서 골라낸다. 서버에 접속하지 않는다.
2. **점검 항목 생성** — `config/profiles.yml`의 `existing-host-drift` profile을 서버별 변수(hostname, IP 등)로 채워 로컬에서 명령 목록을 만든다. 서버에 접속하지 않는다.
3. **점검 명령을 DRY-RUN으로 출력** — 서버별로 실행하면 될 읽기 전용 Ansible 명령을 화면에 보여줄 뿐, 실행하거나 서버에 접속하지 않는다.
4. **복구 명령을 DRY-RUN으로 출력** — 3번에서 차이가 발견된 항목에 한해 복구용 Ansible 명령(`--check --diff` 포함)을 보여준다. 역시 실행하지 않는다.
5. **관리자가 검토 후 직접 실행** — 3, 4번에서 출력된 명령을 관리자가 그대로 복사해 실행해야 서버 상태가 바뀐다. `server-state` 는 이 단계를 대신 실행하지 않는다.

실제 명령은 다음과 같다. 1~3번 단계가 `check` 한 번으로 함께 실행된다.

```
cd /home/jy/server_manage

# 전체 서버의 점검 항목 확인
./server-state/bin/server-state check \
  --hosts all \
  --profile existing-host-drift

# 특정 서버(farm8)만 확인
./server-state/bin/server-state check \
  --hosts farm8 \
  --profile existing-host-drift
```

두 명령 모두 서버에 접속하거나 서버 상태를 바꾸지 않는다. 화면에 출력된 `DRY-RUN` 명령을 관리자가 직접 실행하거나, 별도 자동화가 그 명령을 실행하도록 연결해야 실제 점검(3번 단계 이후)이 이루어진다.

4번 단계, 즉 차이가 있을 때 사용할 복구 명령만 먼저 볼 수도 있다.

```
./server-state/bin/server-state plan \
  --hosts farm8 \
  --profile existing-host-drift
```

2. 신규 서버 구축 흐름

새 서버를 운영 서버와 같은 상태로 만드는 것이 `new-host-bootstrap`의 목적이다. 다만 아무것도 설치되지 않은 장비의 전원 투입부터 전부 처리하는 형태는 아니다. 다음 조건은 먼저 준비되어 있어야 한다.

- Ubuntu가 설치되어 있다.

- IP, hostname과 SSH 접속이 준비되어 있다.
- 관리 계정이 비대화형 sudo를 사용할 수 있다.
- 서버가 공용 inventory에 등록되어 있다.

이후 표준 설치 순서를 확인한다.

```
./server-state/bin/server-state plan \
  --hosts farm8 \
  --profile new-host-bootstrap
```

`ansible_playbook/bootstrap_gpu_server.yml`에는 package 설치와 설정 파일 변경을 실제로 수행하는 idempotent task가 들어 있다. 각 단계는 tag로 나뉘며, 먼저 생성된 명령의 host, tag와 변수를 확인한 뒤 `--check --diff`로 예상 변경을 검토한다. 실제 적용은 검토가 끝난 Ansible 명령에서 `--check`를 제거하여 관리자가 실행한다.

다음 항목은 공통 설정이라도 자동으로 밀어 넣지 않고 별도 승인을 요구한다.

- NVIDIA driver 변경: reboot와 실행 중 GPU workload 조정이 필요하다.
- Kubernetes join: cluster별 token과 controller 승인이 필요하다.
- Kerberos machine keytab: FARM과 LAB의 도메인 가입 방식이 다르고 secret을 안전하게 전달해야 한다.
- Kerberos NFS mount: 실행 중 container와 사용자 session에 영향을 줄 수 있다.

Kerberos/NFS의 상세 점검과 복구 순서는 [Kerberos/NFS 매뉴얼](#)에서 관리한다.

현재 `apply`도 실제 변경 없이 복구 계획만 출력한다.

```
./server-state/bin/server-state apply \
  --hosts farm8 \
  --profile existing-host-drift
```

`apply --execute`는 아직 구현되지 않았으며 실행하면 오류로 중단된다.

3. 공용 inventory와 서버 목록

기본 inventory는 `user-lifecycle/server_info/servers.jsonl`이다. 서버 한 대당 한 줄(JSON)로 host 이름, `server_id`, domain(FARM/LAB), SSH 접속 정보 (`ansible_host`, `ansible_port`, `ansible_user`), 네트워크 인터페이스, OS 버전 등을 담는다. 이 파일은 사람이 손으로 쓰지 않는다. `user-lifecycle/server_info/generate_servers_jsonl.py`가 Ansible `setup` facts(각 서버에서 실제로 수집한 정보)와 고정 topology 정보를 합쳐서 만든 결과물이다.

이 파일은 원래 `user-lifecycle`(사용자·container 생성)이 관리하던 서버 목록이다. `server-state`는 점검할 서버 목록이 필요할 때 이 파일을 그대로 읽어서 쓰고("공용" inventory), 자신만의 목록을 별도로 만들지 않는다. IP, SSH port와 논리 server ID를 두 군데에서 따로 관리하면 서로 달라지는 문제가 생기기 때문이다. `server-state`의 `script/inventory.py`가 이 파일을 읽어서 `--hosts` selector에 맞는 서버를 골라낸다.


```
./server-state/bin/server-state list-hosts --hosts all
./server-state/bin/server-state list-hosts --hosts farm
./server-state/bin/server-state list-hosts --hosts farm8,lab10
```

selector는 `all`, `farm`, `lab`, 개별 host 이름과 여러 host 조합을 지원한다.

4. 공통 설정 추가 방법

1. 설정을 실제로 관리할 모듈을 정한다.
2. `config/profiles.yml` 에 작은 profile을 추가한다.
3. 부작용 없이 읽을 수 있는 항목만 `checks` 에 둔다.
4. 복구는 가능하면 Ansible `--check --diff` 명령부터 제공한다.
5. driver, cluster join, keytab과 mount처럼 위험한 작업은 `manual` 또는 `gated` 로 둔다.
6. 모든 서버에 필요한 설정이면 `new-host-bootstrap` 과 `existing-host-drift` 에 모두 추가한다.
7. profile 순서와 서버별 변수 치환 test를 추가한다.

5. 테스트

```
cd /home/jy/server_manage/server-state
python3 -m unittest discover -s tests -v

./bin/server-state --format json check \
  --hosts farm8 \
  --profile existing-host-drift

./bin/server-state plan \
  --hosts lab10 \
  --profile new-host-bootstrap
```

테스트와 dry-run 출력에서는 실제 운영 서버를 변경하지 않는다.